GitHub Migration Plan

Objective

Migrate from a personal GitHub account to a GitHub Organization while:

- Preserving all repositories and Git history.
- Using Organization Secrets instead of repository secrets.
- Replacing machine GitHub accounts with GitHub Apps.
- Applying the Principle of Least Privilege.
- Maintaining Azure/GitHub Actions integrations.
- Minimizing downtime.

Current State

Personal Account

```
lakshmanachari-panuganti
├── repo1
├── repo2
├── repo3
└── Repository Secrets
```

Additional Accounts

```
omg-ai-developer
devilsadvocate-reviewer
```

These accounts were originally intended for AI development and AI code review.

The goal is to eliminate the need for these accounts by using GitHub Apps.

Target Architecture

```
Personal Account
└── lakshmanachari-panuganti-lab
        │
```

```
        │ Owner
        ▼
Organization
└── lakshmanachari-panuganti
    ├── repo1
    ├── repo2
    ├── repo3
    ├── Organization Secrets
    ├── AI Developer GitHub App
    └── AI Reviewer GitHub App
```

Phase 1 — Rename Personal Account

Rename:

```
lakshmanachari-panuganti
```

to

```
lakshmanachari-panuganti-lab
```

After the rename, verify:

- GitHub login works.
- Profile is accessible.
- Existing repository redirects function correctly.

Phase 2 — Create Organization

Log in using:

```
devilsadvocate-reviewer
```

Create a new GitHub Organization named:

```
lakshmanachari-panuganti
```

After creation:

- Invite `lakshmanachari-panuganti-lab`.
- Promote `lakshmanachari-panuganti-lab` to Organization Owner.

(Optional)

Later remove `devilsadvocate-reviewer` as an owner or member if it is no longer required.

Phase 3 — Transfer Repositories

Transfer every repository from:

```
lakshmanachari-panuganti-lab
```

to

```
Organization
lakshmanachari-panuganti
```

Verify for every repository:

- Branches
- Tags
- Releases
- Issues
- Pull Requests
- Actions workflows
- Branch protection rules
- Repository settings
- Clone URLs
- Redirects

Phase 4 — Migrate Secrets

Inventory every repository secret.

Examples:

- OPENAI_API_KEY

- ANTHROPIC_API_KEY
- GEMINI_API_KEY
- AZURE_CLIENT_ID
- AZURE_CLIENT_SECRET
- AZURE_TENANT_ID
- AZURE_SUBSCRIPTION_ID
- Docker credentials
- Personal Access Tokens
- Other shared credentials

Create Organization Secrets.

Grant access only to repositories that require each secret.

Remove duplicated repository secrets once migration is verified.

Phase 5 — Azure Validation

Inspect every GitHub Actions workflow.

Identify workflows using:

- azure/login
- Azure CLI
- Azure Functions
- Azure Container Registry
- Azure Storage
- Azure Key Vault
- Azure Web Apps
- Bicep
- Terraform
- OIDC authentication

Verify:

- Organization secrets resolve correctly.
- Azure authentication succeeds.
- Deployments continue working.
- OIDC subject claims are updated if repository ownership changes require it.

Phase 6 — Replace Machine Users

Replace:

```
omg-ai-developer
devilsadvocate-reviewer
```

with GitHub Apps.

Create:

- AI Developer App
- AI Reviewer App

or

Create a single GitHub App if both responsibilities can be combined.

GitHub App Permissions

AI Developer

Repository Permissions

Contents

- Read
- Write

Pull Requests

- Read
- Write

Issues

- Read
- Write

Metadata

- Read

Checks

- Read
- Write (optional)

Commit Status

- Read
- Write (optional)

No Repository Administration.

No Organization Administration.

No Repository Deletion.

No Secret Management.

AI Reviewer

Repository Permissions

Contents

- Read

Pull Requests

- Read
- Write

Metadata

- Read

Checks

- Read

Commit Status

- Read

No Push permissions (unless explicitly required).

No Repository Administration.

No Organization Administration.

No Secret Management.

Security Principles

Follow Least Privilege.

GitHub Apps must never receive permissions for:

- Repository deletion
- Repository transfer
- Repository administration
- Organization administration
- Billing
- Member management
- Secret management
- Actions runner management

Administrative permissions remain only with the Organization Owner.

Workflow Validation

For every repository:

- Execute GitHub Actions.
- Validate Azure login.
- Validate deployments.
- Validate organization secrets.
- Validate GitHub App permissions.
- Verify pull request creation.
- Verify code review.
- Verify merge process.

Cleanup

After successful migration:

- Remove obsolete repository secrets.
- Remove unused Personal Access Tokens.
- Remove unused machine-user accounts from the organization.
- Update documentation.
- Update local Git remotes if desired.

Deliverables

Produce a migration report containing:

Repository Summary

- Repository Name
- Migration Status
- Workflow Status
- Secret Status
- Azure Status

GitHub Apps

- Installed repositories
- Permissions
- Authentication method

Organization Secrets

- Secret name
- Repository access
- Validation status

Validation

- Successful workflows
- Failed workflows
- Missing permissions
- Required manual actions

Success Criteria

- All repositories owned by the GitHub Organization.
- Personal account retained as Organization Owner.
- Shared secrets managed as Organization Secrets.
- GitHub Apps replace machine-user accounts.
- Azure deployments continue functioning.
- GitHub Actions pass successfully.
- Least-privilege permissions implemented.
- No duplicated secrets across repositories.
- Complete migration report generated.